

Memory in Toy Transformer Models

This is a work-in-progress draft. Some sections are drafted with the help of Claude and not all claims have been independently verified. Treat numbers as preliminary.

1 Introduction

The goal of this project is to understand how memorised facts are stored in the weights of transformer models. To isolate the mechanisms involved, we study a minimal *toy model*: a single-layer transformer that receives two input tokens and must produce one output token. A *fact* is a specific pair of input tokens mapped to a specific output token. The mapping is random, so the model cannot exploit any systematic structure and must genuinely memorise each fact.

We proceed in two stages. First, by selectively enabling or disabling components of the architecture — attention, feed-forward layer, biases, normalisation, residual connections — we measure which parts of the model matter for fact memorisation. We do this by measuring the maximum number of facts each configuration can learn at a fixed (small) model size. Second, we take a small number of configurations and scale them up, to see how the maximum number of memorised facts grows with model size.

In addition, we attempt to match the trained model’s performance with a hand-coded weight-setting algorithm (i.e. not learned via gradient descent). Why attempt this? Because a good measure of whether you understand something is whether you can replicate it: if we understand how the model stores facts in its weights, we should be able to hand-code a model that does the same thing. Our attempt does not fully succeed, but we think it represents some non-zero progress.

2 Model Architecture

The model, MEMORYTOYMODEL, is a single-layer transformer whose components can be toggled on or off via a set of architectural flags. All configurable options are collected in a MODELSETTINGS object; the full list is summarised in Table 1 and described below.

2.1 Overview

The forward pass proceeds through up to six stages:

1. **Embedding**: The input tokens are mapped into a d_{residual} -dimensional residual stream. The precise mechanism depends on whether attention is enabled. (See Section 2.2.)
2. **Layer Norm 1** (optional): An RMSNorm that normalises the input to the attention layer; the residual stream itself is left unchanged, and the attention output is added back to it. Present if and only if both attention and norms are enabled.
3. **Attention** (optional): If enabled, the embedded sequence is processed by a single self-attention layer; afterwards only the last sequence position is retained. The attention pattern is either learned or fixed to a uniform average. (See Section 2.3.)

4. **Layer Norm 2** (optional): An RMSNorm that normalises the input to the feed-forward block, again leaving the residual stream unchanged. Present if and only if both feed-forward and norms are enabled.
5. **Feed-Forward block** (optional): If enabled, the activations pass through a GELU or ReLU MLP, with an optional residual connection around it. (See Section 2.4.)
6. **Output Head**: A final RMSNorm (if enabled) followed by a linear projection maps the residual stream to logits over the output vocabulary.

Parameter	Values	Description
<i>Data dimensions</i>		
seq_len	2	Number of input tokens per fact
input_vocab_size	32, 64, 128, 256	Size of the input token vocabulary
output_vocab_size	16, 32, 64, 128	Size of the output token vocabulary
n_facts	int	Number of facts to memorise
seed	42	Random seed for fact generation
<i>Internal dimensions</i>		
d_residual	16, 32, 64, 128	Residual-stream (embedding) dimension
n_heads	1	Number of attention heads
d_ff	16, 32, 64, 128	Hidden dimension of the feed-forward block
<i>Architectural toggles</i>		
attention	True/False	Enable self-attention
qk_is_one	True/False	If attention is on, replace the learned QK pattern with a fixed uniform average
ff	True/False	Enable feed-forward block
bias	True/False	Include bias terms in linear layers
norms	True/False	Use RMSNorm (otherwise identity)
ff_residual	True/False	Residual connection around FF block
ff_activation_type	GELU/ReLU	Activation function in the FF block

Table 1: Every configurable setting of MEMORYTOYMODEL, with the value (or values) each one takes across the experiments in this document shown in the **Values** column. The code also accepts other values for the *Data dimensions* and *Internal dimensions* (including seq_len, n_heads and seed); only those listed are actually used.

2.2 Embedding

Two modes are available, controlled by the `attention` flag.

With attention (`attention = True`). Each input token is embedded via a learned token embedding $W_E \in \mathbb{R}^{V_{\text{in}} \times d_{\text{residual}}}$ and summed with a learned positional embedding $W_P \in \mathbb{R}^{T \times d_{\text{residual}}}$, where T is the sequence length and V_{in} the input vocabulary size. The embedded sequence is normalised with RMSNorm (if `norms = True`, otherwise the identity) and passed through a single self-attention layer (Section 2.3). Only the representation at the *last* sequence position is retained, combined with the embedding at the same position by a residual connection that is always active:

$$x = e_T + \text{Attn}(\text{Norm}(e))_T,$$

where $e \in \mathbb{R}^{T \times d_{\text{residual}}}$ is the embedded sequence and subscript T denotes the last position.

Without attention (`attention = False`). A separate embedding matrix $W_{E_i} \in \mathbb{R}^{V_{\text{in}} \times d_{\text{residual}}}$ is learned for each input position $i \in \{1, \dots, T\}$, and the residual-stream vector is their element-wise sum:

$$x = \sum_{i=1}^T W_{E_i} [\text{token}_i].$$

This removes the attention mechanism entirely, replacing it with a simple additive composition of per-position embeddings. As we will see, this replacement is actually *more* powerful for fact memorisation than attention is.

2.3 Attention

When attention is enabled, a single linear projection produces queries, keys, and values (Q, K, V), split across n_{heads} heads of dimension $d_{\text{head}} = d_{\text{residual}}/n_{\text{heads}}$. The new flag `qk_is_one` selects between two regimes for the attention pattern:

Full attention (`qk_is_one = False`). The usual scaled dot-product causal attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M\right) V,$$

where M is a causal mask. The attention weights depend on the input through Q and K .

Uniform attention (`qk_is_one = True`). The attention matrix is replaced by a fixed uniform pattern $\frac{1}{T}\mathbf{1}\mathbf{1}^\top$, so each position receives the average of all value vectors regardless of content. The Q and K projections are then unused; only the value and output projections are learned. This isolates “a learned linear mixing of the tokens” from “a learned, content-dependent routing of the tokens”.

In both cases the head outputs are concatenated and projected back to d_{residual} dimensions by a learned linear map, with no biases in the attention projections.

2.4 Feed-Forward Block

When enabled (`ff = True`), the feed-forward block is a standard two-layer MLP:

$$\text{FF}(x) = W_2 \sigma(W_1 x + b_1) + b_2,$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{residual}}}$, $W_2 \in \mathbb{R}^{d_{\text{residual}} \times d_{\text{ff}}}$, and σ is GELU or ReLU. Biases b_1, b_2 are included only when `bias = True`. The input to the MLP is optionally normalised with RMSNorm (controlled by `norms`). A residual connection around the block is controlled by `ff_residual`:

$$x \leftarrow \begin{cases} x + \text{FF}(\text{Norm}(x)) & \text{if } \text{ff_residual} = \text{True}, \\ \text{FF}(\text{Norm}(x)) & \text{otherwise.} \end{cases}$$

2.5 Output Head

The residual-stream vector is passed through a final RMSNorm (if `norms = True`) and then a linear projection $W_H \in \mathbb{R}^{V_{\text{out}} \times d_{\text{residual}}}$ (with bias when `bias = True`) to produce logits over the output vocabulary of size V_{out} .

3 Fact Generation

Facts are generated deterministically from the random seed. Each fact consists of `seq_len` input tokens and a target token. For `seq_len = 2`, the input pairs are drawn without replacement from all V_{in}^2 ordered pairs, and the target is assigned cyclically from the output vocabulary ($\text{target}_i = i \bmod V_{\text{out}}$). This gives a near-uniform distribution over output classes while keeping the input-output mapping arbitrary, so the model cannot learn any simple rule. Note that for $V_{\text{in}} = 32$, `seq_len = 2` there are only $32^2 = 1024$ possible distinct facts, which is the absolute ceiling on `n_facts`.

4 Training

The model is trained full-batch with the Adam optimiser, using *cross-entropy* (CE) loss over the output vocabulary. We track **best-guess accuracy**: the fraction of facts for which $\arg \max$ of the logits equals the correct target. Training uses early stopping: it halts when best-guess accuracy reaches 1.0, or when it has not improved for `patience` epochs.

Capacity search. To find the maximum number of facts a configuration can memorise, we binary-search over `n_facts`. For each candidate count we train several models from fresh random initialisations, optionally cycling through a list of learning rates when progress stalls. A candidate count counts as “learnable” according to one of three criteria over the `number_of_attempts` repeats:

- **any**: at least one attempt reaches perfect accuracy;
- **most**: a majority of attempts reach perfect accuracy;
- **all**: every attempt reaches perfect accuracy.

The binary search narrows until the remaining interval is below `precision`. The two criteria used in the experiments below, *any* and *most*, bracket the capacity: *any* reports the most optimistic “best case with a lucky seed”, while *most* reports a more robust “typically achievable” capacity. By construction the *any* number is always at least the *most* number.

5 Experiment: Which architectural components matter for fact memorisation?

In this experiment we fix a small model size (Table 2) and sweep over the architectural toggles, measuring the maximum number of facts each configuration can memorise. The results are in Table 4.

Each architecture is measured under three criteria for counting a fact count as “learnable”: *any* (at least one of 11 attempts reaches perfect accuracy), *majority* (more than half do), and *all* (every attempt does). Each criterion is repeated four times with independently sampled attempts, giving twelve runs that serve as a consistency check. Every measurement uses 11 attempts per candidate fact count, `n_epochs = 50 000` with early stopping, and the learning-rate schedule $\text{lr} = [10^{-2}, 3 \times 10^{-3}, 10^{-3}]$. The full per-run settings are listed in Table 3.

The three criteria are not just noise reduction — they probe different things. *Any* measures the best capacity reachable with a lucky initialisation; *all* measures the capacity the model reaches *reliably*, on every seed. The gap between them is a measure of how fragile a configuration is to train, and below it turns out to be at least as informative as the capacities themselves.

Two features of Table 4 therefore read as *stability*. When the *any*, *majority*, and *all* values of a row are close, training is stable: the architecture reliably learns that many facts, because nearly

every seed reaches the same capacity. When instead the value falls steeply from *any* to *majority* to *all*, and the row carries many boxed outliers (repeats more than 20% from their group’s median), training is unstable: the architecture only occasionally reaches a high capacity and usually lands well short of it, so it does not reliably get to, or even near, its best capacity. These two signals do coincide: across the 54 architectures the number of outliers in a row correlates strongly with its drop from *any* to *all* (Pearson 0.77, Spearman 0.81),¹ the mean drop rising monotonically from about 17% for rows with no outliers to over 85% for the most outlier-heavy rows.

Reliability. Individual training runs occasionally fail, producing spuriously low capacity estimates. Running 11 attempts and reporting the three criteria largely suppresses these: within each criterion the four repeats agree closely for the *None* and *Uniform* rows (usually within a few percent). The *Full*-attention rows remain noisier between repeats, which is itself consistent with full attention being the hardest variant to train.

Data and model dimension parameters	Value
input_vocab_size	32
output_vocab_size	16
d_residual	16
d_ff	16

Table 2: Fixed model and data dimensions for the experiment in Section 5. The architectural toggles are varied across rows of Table 4. With these dimensions the maximum possible number of facts is $32^2 = 1024$.

Parameter	Value
loss_type	CE
n_epochs	50 000 (with early stopping)
lr	$[10^{-2}, 3 \times 10^{-3}, 10^{-3}]$
patience	5 000
target_accuracy	best_guess_accuracy
threshold_to_continue	0.95
precision	8
number_of_attempts	11
any_all_most	<i>any, majority, all</i>

Table 3: Training and capacity-search settings shared by all runs in Table 4. The **Max Facts** columns of Table 4 are the twelve runs: four independent repeats of each of the *any*, *majority*, and *all* criteria.

Two patterns recur across almost every comparison below. First, each component we vary has a fairly consistent “better” setting that wins on the large majority of architectures. Second, and more telling, each component matters far more for *reliability* than for *peak* capacity: the average gap between the better and worse setting grows steadily from *any* to *majority* to *all*. A toggle

¹Both are correlation coefficients in $[-1, +1]$ measuring how strongly two quantities move together (+1 perfectly together, 0 unrelated, −1 opposite). For paired samples (x_i, y_i) with means $\bar{x}, \bar{y}$, the *Pearson* coefficient measures a linear relationship, $r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$. The *Spearman* coefficient applies the same formula to the *ranks* of the values, capturing any monotonic (not just linear) trend and remaining robust to outliers; with no tied ranks it reduces to $1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)}$, where d_i is the difference between the ranks of x_i and y_i . See Wikipedia: Pearson correlation coefficient and Spearman’s rank correlation coefficient.

Attn	FF	Norm	Res	Bias	Act	Any of 11				Majority of 11				All of 11			
						(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)
None	✓	✗	✗	✗	GELU	776	760	784	896	688	672	704	704	640	560	600	560
None	✓	✗	✗	✗	ReLU	664	688	736	816	568	568	504	576	416	<u>184</u>	<u>448</u>	288
None	✓	✗	✗	✓	GELU	864	848	896	904	776	800	784	792	680	680	680	720
None	✓	✗	✗	✓	ReLU	960	824	872	872	680	720	688	712	488	472	528	496
None	✓	✗	✓	✗	GELU	992	984	984	1008	952	960	952	960	920	928	912	928
None	✓	✗	✓	✗	ReLU	976	960	968	976	928	920	920	936	912	888	912	896
None	✓	✗	✓	✓	GELU	1000	1008	992	1008	952	968	960	960	920	920	920	920
None	✓	✗	✓	✓	ReLU	984	976	1000	976	936	928	944	920	904	904	928	912
None	✓	✓	✗	✗	GELU	888	888	920	960	824	856	832	824	792	792	776	800
None	✓	✓	✗	✗	ReLU	472	560	552	576	328	328	<u>248</u>	344	<u>256</u>	168	<u>88</u>	168
None	✓	✓	✗	✓	GELU	1024	1024	1024	1024	1024	1024	1024	1024	984	952	1000	984
None	✓	✓	✗	✓	ReLU	1024	1024	1024	1024	880	888	976	920	528	480	536	536
None	✓	✓	✓	✗	GELU	1024	1024	1024	1024	1000	1016	1024	1024	984	992	984	952
None	✓	✓	✓	✗	ReLU	1024	1024	1024	1024	1016	1024	1024	1024	1008	984	992	992
None	✓	✓	✓	✓	GELU	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024	1000	1024
None	✓	✓	✓	✓	ReLU	1024	1024	1024	1024	1024	1024	1024	1024	1008	1024	1024	1008
None	✗	✗	N/A	N/A	N/A	208	208	208	208	208	208	208	208	208	208	208	208
None	✗	✓	N/A	N/A	N/A	576	576	576	552	536	544	528	544	496	480	472	504
Unif	✓	✗	✗	✗	GELU	696	680	680	688	576	536	552	552	440	424	480	464
Unif	✓	✗	✗	✗	ReLU	640	640	616	672	472	512	480	512	336	376	368	392
Unif	✓	✗	✗	✓	GELU	712	696	720	720	664	640	632	672	528	520	488	520
Unif	✓	✗	✗	✓	ReLU	712	712	704	720	640	640	584	640	448	440	480	432
Unif	✓	✗	✓	✗	GELU	832	824	824	816	792	792	800	792	768	720	744	760
Unif	✓	✗	✓	✗	ReLU	816	808	816	800	760	776	784	784	736	744	728	<u>504</u>
Unif	✓	✗	✓	✓	GELU	824	832	840	840	800	808	800	808	728	760	776	752
Unif	✓	✗	✓	✓	ReLU	824	792	816	800	784	776	784	784	760	768	768	760
Unif	✓	✓	✗	✗	GELU	800	800	776	784	744	736	744	736	696	696	696	704
Unif	✓	✓	✗	✗	ReLU	496	504	528	440	280	296	312	248	104	144	104	<u>160</u>
Unif	✓	✓	✗	✓	GELU	880	904	888	888	848	848	848	840	776	760	776	768
Unif	✓	✓	✗	✓	ReLU	920	888	896	880	768	728	768	768	240	248	248	<u>448</u>
Unif	✓	✓	✓	✗	GELU	880	896	912	920	856	864	848	856	832	816	800	792
Unif	✓	✓	✓	✗	ReLU	872	880	904	872	856	840	848	856	800	792	800	808
Unif	✓	✓	✓	✓	GELU	944	936	960	960	912	904	904	896	856	864	880	856
Unif	✓	✓	✓	✓	ReLU	912	944	936	960	896	888	888	872	816	832	832	832
Unif	✗	✗	N/A	N/A	N/A	208	208	208	208	200	192	208	192	192	192	192	192
Unif	✗	✓	N/A	N/A	N/A	416	408	408	408	376	376	376	376	312	304	328	336
Full	✓	✗	✗	✗	GELU	536	584	504	600	<u>304</u>	392	384	384	<u>136</u>	120	<u>80</u>	88
Full	✓	✗	✗	✗	ReLU	456	576	488	584	312	352	272	312	<u>112</u>	<u>64</u>	<u>128</u>	<u>48</u>
Full	✓	✗	✗	✓	GELU	600	632	696	616	336	384	312	352	<u>144</u>	<u>152</u>	<u>24</u>	<u>48</u>
Full	✓	✗	✗	✓	ReLU	544	656	536	640	280	272	368	360	<u>144</u>	<u>16</u>	72	56
Full	✓	✗	✓	✗	GELU	688	664	656	640	344	328	384	<u>248</u>	<u>56</u>	<u>128</u>	<u>136</u>	<u>40</u>
Full	✓	✗	✓	✗	ReLU	<u>488</u>	672	704	688	336	320	248	248	88	<u>112</u>	64	72
Full	✓	✗	✓	✓	GELU	624	664	712	608	264	296	248	328	<u>16</u>	<u>128</u>	<u>96</u>	<u>56</u>
Full	✓	✗	✓	✓	ReLU	648	600	504	592	272	248	<u>176</u>	<u>512</u>	96	<u>64</u>	80	<u>112</u>
Full	✓	✓	✗	✗	GELU	784	768	768	696	648	656	600	616	264	368	232	232
Full	✓	✓	✗	✗	ReLU	440	512	496	472	312	296	320	328	128	<u>192</u>	152	136
Full	✓	✓	✗	✓	GELU	800	896	800	760	600	672	<u>504</u>	704	336	<u>184</u>	376	296
Full	✓	✓	✗	✓	ReLU	800	776	728	832	440	520	576	536	<u>152</u>	232	264	288
Full	✓	✓	✓	✗	GELU	864	776	832	808	632	704	736	704	232	216	<u>320</u>	184
Full	✓	✓	✓	✗	ReLU	776	832	832	832	680	688	728	688	280	240	280	296
Full	✓	✓	✓	✓	GELU	848	816	872	808	720	696	632	704	320	<u>248</u>	384	<u>448</u>
Full	✓	✓	✓	✓	ReLU	832	840	840	792	768	704	640	632	336	<u>208</u>	320	240
Full	✗	✗	N/A	N/A	N/A	208	168	224	208	112	120	88	96	16	16	16	<u>48</u>
Full	✗	✓	N/A	N/A	N/A	384	<u>272</u>	416	384	168	176	<u>136</u>	192	64	48	56	56

Table 4: Maximum facts memorised per architectural configuration ($d_{\text{residual}} = 16$, $d_{\text{ff}} = 16$, CE loss). **Attn** is the token-mixing mechanism: *None* (additive per-position embeddings), *Unif* (uniform/averaging attention, `qk_is_one = True`), or *Full* (learned attention). The *Any* / *Majority* / *All* column groups apply the corresponding criterion (a fact count counts as learned if at least one / more than half / all of the 11 attempts reach perfect accuracy); the four numbered columns within each group are independent repeats. Within each group the row-wise maximum is shown in **bold** and values more than 20% from the group’s median are boxed as outliers. For `ff = False` rows the residual, bias and activation flags do not apply (N/A). Note that 1024 is the dataset ceiling, so configurations reaching it have saturated the data rather than the model.

that moves peak capacity by a few percent can move the *reliably*-memorised count by tens or hundreds of percent. We take the components in roughly increasing order of importance, ending with the two that dominate — the feed-forward block and the attention mechanism.

5.1 Activation function: a small edge for GELU, mostly about reliability

The activation function has the smallest effect on peak capacity of any toggle. Under *any*, GELU beats ReLU on 19 of the 24 **ff = True** architectures, but by a mean of only +11%, and several pairs tie (Table 5). The gap then widens sharply with the criterion — +25% under *majority*, +72% under *all*. This is the cleanest illustration of the reliability pattern: GELU and ReLU reach similar *best-case* capacities, but ReLU much more often fails to memorise the same facts on *every* seed. The few exceptions (a handful of *Full*-attention rows where ReLU edges ahead, down to −13%) are small and look like training noise. We therefore treat activation as the least important choice for capacity — it matters mainly for training stability.

				Any of 11			Majority of 11			All of 11					
Attn	Norm	Res	Bias	G>R	=	R>G	Mean %Δ	G>R	=	R>G	Mean %Δ	G>R	=	R>G	Mean %Δ
None	✗	✗	✗	4			+10.9%	4			+25.3%	4			+96.6%
None	✗	✗	✓	3		1	−0.2%	4			+12.6%	4			+39.3%
None	✗	✓	✗	4			+2.3%	4			+3.2%	3	1		+2.2%
None	✗	✓	✓	3		1	+1.8%	4			+3.0%	3		1	+0.9%
None	✓	✗	✗	4			+70.0%	4			+171.8%	4			+434.7%
None	✓	✗	✓		4		+0.0%	4			+12.0%	4			+88.7%
None	✓	✓	✗		4		+0.0%		2	2	−0.6%	1		3	−1.6%
None	✓	✓	✓		4		+0.0%		4		+0.0%	2	1	1	+0.2%
Unif	✗	✗	✗	4			+6.9%	4			+12.4%	4			+23.1%
Unif	✗	✗	✓	1	2	1	+0.0%	3	1		+4.2%	4			+14.5%
Unif	✗	✓	✗	4			+1.7%	4			+2.3%	3		1	+13.5%
Unif	✗	✓	✓	3		1	+3.2%	4			+2.8%	1		3	−1.3%
Unif	✓	✗	✗	4			+61.3%	4			+162.4%	4			+465.4%
Unif	✓	✗	✓	2		2	−0.6%	4			+11.7%	4			+178.5%
Unif	✓	✓	✗	4			+2.3%	1	3		+0.7%	2	1	1	+1.3%
Unif	✓	✓	✓	2	1	1	+1.3%	4			+2.0%	4			+4.4%
Full	✗	✗	✗	4			+6.2%	3		1	+18.3%	3		1	+38.7%
Full	✗	✗	✓	2		2	+8.2%	2		2	+10.9%	1	1	2	+192.3%
Full	✗	✓	✗	1		3	+6.5%	3	1		+14.9%	2		2	+11.5%
Full	✗	✓	✓	3		1	+12.7%	2		2	+5.3%	2		2	−3.3%
Full	✓	✗	✗	4			+57.6%	4			+101.2%	4			+80.3%
Full	✓	✗	✓	2	1	1	+4.2%	3		1	+21.1%	3		1	+36.4%
Full	✓	✓	✗	1	1	2	+0.4%	3		1	−0.3%	1		3	−12.7%
Full	✓	✓	✓	3		1	+1.2%	1		3	+0.7%	3		1	+30.3%

Table 5: GELU vs. ReLU, one row per **ff = True** architecture (the 24 configurations that differ only in activation function). For each criterion, **G>R**, **=**, and **R>G** count the repeats (out of 4) in which GELU exceeds, equals, or trails ReLU, paired by repeat index; a count of 4 (all repeats agree) is shown in **bold**. **Mean %Δ** is the mean of (GELU − ReLU)/ReLU over the repeats, coloured **green** when positive (GELU ahead) and **red** when negative.

5.2 Bias terms: a small but consistent gain

Including bias terms in the linear layers is a modest, consistent help: bias wins on 19–20 of the 24 architectures under every criterion, by a mean of +15% (*any*), +24% (*majority*) and +28% (*all*); these numbers are read off the main sweep (Table 4), as bias gets no separate table. Bias is the one exception to the reliability pattern — its advantage barely grows as the criterion

tightens, and its +28% under *all* is the *smallest* strict-criterion effect of any component. So bias supplies a small uniform lift rather than a reliability multiplier. This even flips its ranking against the activation function: under *any*, bias matters slightly more than GELU-vs-ReLU (+15% vs +11%), but under *all* it matters far less (+28% vs +72%). A few architectures do marginally better without bias, and we expect the whole effect to shrink as the model is scaled up.

5.3 Normalisation: a large gain, and essential without the feed-forward block

RMSNorm is clearly beneficial, and uniquely so in one regime. With the feed-forward block present, norm-on wins on 21–22 of the 24 architectures, by a mean of +14% (*any*), +40% (*majority*) and +120% (*all*) — the now-familiar small-peak, large-reliability shape (Table 6). When the feed-forward block is *absent* (the bottom rows of the table), normalisation becomes essential rather than merely helpful: it roughly doubles capacity on every criterion (+80% to +190%), and, unusually, the benefit is already large under *any* rather than only under *all*. The main genuine break from the pattern is a handful of architectures that prefer no norm, including one strict-criterion row at −65%.

Attn	FF	Res	Bias	Act	Any of 11				Majority of 11				All of 11			
					On	=	Off	Mean %Δ	On	=	Off	Mean %Δ	On	=	Off	Mean %Δ
None	✓	✗	✗	GELU	4			+13.9%	4			+20.6%	4			+34.3%
None	✓	✗	✗	ReLU			4	−25.5%			4	−43.9%			4	−42.3%
None	✓	✗	✓	GELU	4			+16.7%	4			+30.0%	4			+42.1%
None	✓	✗	✓	ReLU	4			+16.5%	4			+31.0%	4			+4.9%
None	✓	✓	✗	GELU	4			+3.2%	4			+6.3%	4			+6.1%
None	✓	✓	✗	ReLU	4			+5.6%	4			+10.4%	4			+10.2%
None	✓	✓	✓	GELU	4			+2.2%	4			+6.7%	4			+10.7%
None	✓	✓	✓	ReLU	4			+4.1%	4			+9.9%	4			+11.4%
None	✗	N/A	N/A	N/A	4			+174.0%	4			+158.7%	4			+134.6%
Unif	✓	✗	✗	GELU	4			+15.2%	4			+33.6%	4			+54.8%
Unif	✓	✗	✗	ReLU			4	−23.1%			4	−42.4%			4	−65.4%
Unif	✓	✗	✓	GELU	4			+25.0%	4			+29.8%	4			+50.0%
Unif	✓	✗	✓	ReLU	4			+25.9%	4			+21.3%	1		3	−33.7%
Unif	✓	✓	✗	GELU	4			+9.5%	4			+7.8%	4			+8.4%
Unif	✓	✓	✗	ReLU	4			+8.9%	4			+9.6%	4			+21.3%
Unif	✓	✓	✓	GELU	4			+13.9%	4			+12.4%	4			+14.6%
Unif	✓	✓	✓	ReLU	4			+16.1%	4			+13.3%	4			+8.4%
Unif	✗	N/A	N/A	N/A	4			+97.1%	4			+90.1%	4			+66.7%
Full	✓	✗	✗	GELU	4			+36.5%	4			+74.3%	4			+163.6%
Full	✓	✗	✗	ReLU	1		3	−8.0%	2	1	1	+1.7%	4			+104.1%
Full	✓	✗	✓	GELU	4			+28.4%	4			+78.8%	4			+534.4%
Full	✓	✗	✓	ReLU	4			+32.8%	4			+63.4%	4			+509.1%
Full	✓	✓	✗	GELU	4			+23.9%	4			+118.5%	4			+219.6%
Full	✓	✓	✗	ReLU	4			+30.5%	4			+147.1%	4			+245.3%
Full	✓	✓	✓	GELU	4			+28.5%	4			+144.3%	4			+748.4%
Full	✓	✓	✓	ReLU	4			+42.2%	4			+163.3%	4			+222.3%
Full	✗	N/A	N/A	N/A	4			+79.2%	4			+62.8%	4			+191.7%

Table 6: Norm on vs. off, one row per **ff = True** architecture (the 24 configurations that differ only in the RMSNorm layers). For each criterion, **On** and **Off** count the repeats (out of 4) in which norm-on / norm-off has more facts and **=** counts ties, all paired by repeat index; a count of 4 (all repeats agree) is shown in **bold**. **Mean %Δ** is the mean of (on − off)/off over the repeats, coloured **green** when positive (norm helps) and **red** when negative.

5.4 The residual connection: a large, reliable gain

A residual connection around the feed-forward block is one of the strongest beneficial toggles. Residual-on wins on 19–22 of the 24 architectures, by a mean of +21% (*any*), +40% (*majority*) and +100% (*all*), with the largest single-architecture gap reaching +574% under *all* (Table 7). The reliability pattern is again pronounced: the residual connection roughly doubles the number of *reliably* memorised facts. Only one or two architectures prefer no residual, and only marginally.

Attn	Norm	Bias	Act	Any of 11				Majority of 11				All of 11			
				On	=	Off	Mean %Δ	On	=	Off	Mean %Δ	On	=	Off	Mean %Δ
None	✗	✗	GELU	4			+23.8%	4			+38.2%	4			+56.8%
None	✗	✗	ReLU	4			+34.4%	4			+67.6%	4			+204.1%
None	✗	✓	GELU	4			+14.2%	4			+21.8%	4			+33.4%
None	✗	✓	ReLU	4			+11.9%	4			+33.2%	4			+84.1%
None	✓	✗	GELU	4			+12.2%	4			+21.8%	4			+23.8%
None	✓	✗	ReLU	4			+90.8%	4			+233.1%	4			+574.3%
None	✓	✓	GELU		4		+0.0%		4		+0.0%	3	1		+3.9%
None	✓	✓	ReLU		4		+0.0%	4			+12.0%	4			+95.8%
Unif	✗	✗	GELU	4			+20.1%	4			+43.4%	4			+65.8%
Unif	✗	✗	ReLU	4			+26.3%	4			+57.3%	4			+85.8%
Unif	✗	✓	GELU	4			+17.2%	4			+23.4%	4			+46.9%
Unif	✗	✓	ReLU	4			+13.5%	4			+25.1%	4			+70.0%
Unif	✓	✗	GELU	4			+14.2%	4			+15.7%	4			+16.1%
Unif	✓	✗	ReLU	4			+80.0%	4			+201.6%	4			+548.4%
Unif	✓	✓	GELU	4			+6.8%	4			+6.9%	4			+12.2%
Unif	✓	✓	ReLU	3		1	+4.7%	4			+17.0%	4			+199.2%
Full	✗	✗	GELU	4			+19.7%	1	1	2	−9.6%	2		2	−9.2%
Full	✗	✗	ReLU	4			+21.4%	1		3	−7.7%	2		2	+13.4%
Full	✗	✓	GELU	3		1	+2.5%			4	−17.9%	2		2	+53.0%
Full	✗	✓	ReLU	1		3	−0.7%	1		3	−5.4%	3		1	+94.4%
Full	✓	✗	GELU	4			+8.9%	3		1	+10.5%	1		3	−9.0%
Full	✓	✗	ReLU	4			+70.7%	4			+121.9%	4			+86.4%
Full	✓	✓	GELU	3		1	+3.1%	3	1		+12.2%	3		1	+20.9%
Full	✓	✓	ReLU	3		1	+5.7%	4			+34.7%	2		2	+28.8%

Table 7: Residual on vs. off, one row per **ff** = **True** architecture (the 24 configurations that differ only in the residual connection around the FF block). For each criterion, **On** and **Off** count the repeats (out of 4) in which residual-on / residual-off has more facts and = counts ties, all paired by repeat index; a count of 4 (all repeats agree) is shown in **bold**. **Mean** %Δ is the mean of (on − off)/off over the repeats, coloured green when positive (residual helps) and red when negative.

5.5 The feed-forward block: where the facts are stored

The feed-forward block is the single largest effect in the sweep. We can switch it off for only six architectures (the **ff** = **False** runs vary attention and normalisation, holding the residual and bias on and the activation at ReLU), and there adding the block multiplies capacity by a mean of +200% to +270%, helping in every case (Table 8); without it the model tops out at a few hundred facts, with it at 700–1024 (Table 4). This is expected — the MLP is the model’s associative memory, where the facts are actually written — so it sets the overall scale that every other component merely modulates. Its value is largest exactly where the rest of the model is weakest: the gain is biggest with no normalisation (e.g. +373% for *None* without a norm) and shrinks, while staying large, once a norm is present. It is also the one setting under which normalisation flips from helpful to indispensable (Section 5.3).

Attn	Norm	Any of 11				Majority of 11				All of 11			
		On	=	Off	Mean % Δ	On	=	Off	Mean % Δ	On	=	Off	Mean % Δ
None	✗	4			+373.1%	4			+348.1%	4			+338.5%
None	✓	4			+79.7%	4			+90.4%	4			+108.4%
Unif	✗	4			+288.5%	4			+295.4%	4			+297.9%
Unif	✓	4			+128.8%	4			+135.6%	4			+159.1%
Full	✗	4			+194.6%	4			+195.7%	4			+333.3%
Full	✓	4			+133.4%	4			+314.2%	4			+389.6%

Table 8: Feed-forward block on vs. off, for the six (**attention**, **norm**) architectures where the block can be removed (the residual and bias are held on and the activation at ReLU, the settings present on the `ff = False` runs). For each criterion, **On** and **Off** count the repeats (out of 4) in which FF-on / FF-off has more facts and = counts ties; a count of 4 is shown in **bold**. **Mean % Δ** is the mean of $(\text{on} - \text{off})/\text{off}$ over the repeats, coloured green when positive.

5.6 Attention: the more machinery, the fewer facts (None > Unif > Full)

The attention mechanism gives the most striking — and most counterintuitive — result. Across all 16 `ff = True` architectures (and the no-FF rows), capacity is ordered *None* > *Uniform* > *Full* with almost no exceptions (Table 9): the more attention machinery we add, the *fewer* facts the model memorises. As everywhere, the gaps widen under stricter criteria — but here very unevenly, and the unevenness is the interesting part. The *None*→*Uniform* step is a small, stable cost (None beats Uniform by ~ 16 – 26% across all three criteria). The expensive step is *Uniform*→*Full*: replacing the fixed average with a learned softmax barely dents peak capacity (+18% for Uniform over Full under *any*) but is ruinous for reliability (+504% under *all*). The penalty is concentrated almost entirely in the *learned QK* routing, not in having attention at all.

Why would more machinery hurt? It makes sense once you look at what each variant has to learn. The “no attention” path learns a separate embedding table per input position and simply adds them; for two input tokens this is an unconstrained learned function of each token, a very flexible way to write facts into the residual stream. Uniform attention is more restricted (a single shared value projection, averaged over positions), and full learned attention is more restricted still, because the content-dependent softmax routing has to be *learned* and consumes capacity rather than adding it. For pure memorisation the simplest mixing wins; a content-dependent softmax adds little capacity and a great deal of training difficulty — which is exactly the reliability cost we see appear in the *Uniform*→*Full* step.

6 Experiment: How does Max Facts scale with model size?

(Data not yet available — to be written once the scaling runs are complete.)

Any of 11										Majority of 11						All of 11												
Norm	FF	Res	Bias	Act	N>U	Δ	U>F	Δ	N>F	Δ	N>U	Δ	U>F	Δ	N>F	Δ	N>U	Δ	U>F	Δ	N>F	Δ	N>U	Δ	U>F	Δ	N>F	Δ
✗	✓	✗	✗	GELU	4	+17.2%	4	+24.0%	4	+45.0%	4	+25.0%	4	+53.4%	4	+91.1%	4	+30.8%	4	+351.0%	4	+480.9%						
✗	✓	✗	✗	ReLU	4	+13.0%	4	+23.2%	4	+38.9%	4	+12.2%	4	+59.3%	4	+78.3%	2	-8.0%	4	+397.9%	4	+302.2%						
✗	✓	✗	✓	GELU	4	+23.3%	4	+12.3%	4	+38.4%	4	+20.9%	4	+89.4%	4	+128.9%	4	+34.3%	4	+856.4%	4	+1213.2%						
✗	✓	✗	✓	ReLU	4	+23.9%	4	+20.8%	4	+50.3%	4	+12.0%	4	+100.1%	4	+123.1%	4	+10.3%	4	+1024.8%	4	+1127.0%						
✗	✓	✓	✗	GELU	4	+20.4%	4	+24.5%	4	+50.0%	4	+20.4%	4	+149.8%	4	+201.1%	4	+23.3%	4	+995.2%	4	+1239.6%						
✗	✓	✓	✓	ReLU	4	+19.8%	4	+29.9%	4	+55.6%	4	+19.3%	4	+175.2%	4	+228.0%	4	+36.6%	4	+734.5%	4	+1024.7%						
✗	✓	✓	✓	GELU	4	+20.2%	4	+28.4%	4	+54.3%	4	+19.4%	4	+186.2%	4	+241.9%	4	+22.1%	4	+1723.7%	4	+2167.5%						
✗	✓	✓	✓	ReLU	4	+21.8%	4	+39.1%	4	+69.4%	4	+19.2%	4	+199.9%	4	+258.6%	4	+19.4%	4	+807.6%	4	+982.1%						
✓	✓	✗	✗	GELU	4	+15.8%	4	+5.0%	4	+21.7%	4	+12.7%	4	+17.6%	4	+32.5%	4	+13.2%	4	+164.1%	4	+198.6%						
✓	✓	✗	✗	ReLU	3	+10.4%	2	+2.7%	4	+12.5%	3	+11.5%	4	-9.3%	3	-0.4%	3	+38.1%	1	-14.4%	2	+17.2%						
✓	✓	✗	✓	GELU	4	+15.1%	4	+9.7%	4	+26.3%	4	+21.0%	4	+38.8%	4	+67.9%	4	+27.3%	4	+177.5%	4	+252.2%						
✓	✓	✗	✓	ReLU	4	+14.3%	4	+14.6%	4	+30.9%	4	+20.9%	4	+47.8%	4	+78.0%	4	+87.3%	3	+28.6%	4	+135.9%						
✓	✓	✓	✗	GELU	4	+13.6%	4	+10.2%	4	+25.1%	4	+18.7%	4	+23.7%	4	+46.8%	4	+20.8%	4	+254.2%	4	+327.1%						
✓	✓	✓	✓	ReLU	4	+16.1%	4	+7.9%	4	+25.3%	4	+20.2%	4	+22.2%	4	+46.9%	4	+24.3%	4	+193.6%	4	+264.9%						
✓	✓	✓	✓	GELU	4	+7.8%	4	+13.7%	4	+22.6%	4	+13.3%	4	+31.7%	4	+49.2%	4	+17.9%	4	+159.0%	4	+205.5%						
✓	✓	✓	✓	ReLU	4	+9.2%	4	+13.7%	4	+24.0%	4	+15.6%	4	+29.9%	4	+50.2%	4	+22.7%	4	+212.4%	4	+283.1%						
✗	✗	N/A	N/A	N/A		+0.0%	1	+4.2%	1	+4.2%	3	+5.2%	4	+93.7%	4	+103.0%	4	+8.3%	4	+900.0%	4	+983.3%						
✓	✗	N/A	N/A	N/A	4	+39.0%	3	+15.7%	4	+61.0%	4	+43.1%	4	+127.4%	4	+224.9%	4	+52.7%	4	+476.6%	4	+779.5%						

Table 9: Pairwise comparison of the attention variants **None** / **Unif** / **Full**, one row per architecture (the 16 **ff** = **True** rows plus the 2 **ff** = **False** rows, where only normalisation varies). For each criterion and each pair, the **X>Y** column counts the repeats (out of 4) in which variant X beats variant Y (paired by repeat index; a count of 4 is **bold**, zeros blank), and the Δ column is the mean of $(X - Y)/Y$ over the repeats, coloured **green** when positive and **red** when negative.